

プログラミング入門2

第1回

導入、型と演算、条件分岐、式の評価

篠埜 功

講義情報

- 担当教員

- 篠埜 功
- 居室 豊洲校舎 14階 14K32
- E-mail: sasano@sic.shibaura-it.ac.jp

- 講義用web page

<http://www.sic.shibaura-it.ac.jp/~sasano/lecture/lecture.html>

ここへ毎回、前日までに講義用資料を置く。

- 時間・場所

- 火曜日 3～4限
- 2号館PC実習室4,5 および zoom
- 13:20-13:30はホームルームの時間とする。

本講義(演習)の目的

- 現在世界で広く用いられているC言語に触れる。
 - C言語を知っていればJava, C++, C#等、Cの影響を受けている言語の理解にも役立つ。
- C言語を通して、プログラミングをする際に必要な考え方を身につける。
- C言語のすべてを扱うには時間が足りないので、C言語の核となる部分のみを扱う。
- 扱わなかった内容は各自で教科書等を読んで補う。(他人の書いたプログラムを読むためには、あらゆる構文に対応できる必要がある。必要に応じて補う。)

授業の進め方

- 内容説明(3限)
- 演習課題(4限) ---第11回までは毎回、基本課題を2問ずつ出題する(1問0.5点)。できた人はTAに確認してもらう。その日に終わらなかったら次の回に確認してもらう。次回以降でも受け付けるが、なるべく早めに終わらせる。(中間試験があるので、この部分は10回分で合計10点。)
- 第12回,13回は総合演習とする。基本課題を3問出題する(1問1点、計3点)。
- 演習の基本課題の合計得点は13点。(端数が出た場合は切り上げる。)
- 基本課題ができて時間に余裕がある人は発展課題を解いてTAに確認してもらう。発展課題は1問0.5点(総合演習の発展課題は1問1点)で60点を超えない範囲で加点する。
- 演習問題の解答の提出締め切りは第13回の日々の23:59とする。

注意事項

- 課題を他の人と一緒に解いたり、他の人に教えてもらったりしても良いですが、他の人のプログラムを写してそのまま提出した場合、不正行為となります。(写した人も写させた人も)
- 他の人と一緒に同じプログラムを書いた場合は、自分で一から書き直してから提出してください。
- 他の人に自分が書いたプログラムを(slackやメールで)渡すのはやめてください。

参考書(シラバス記載のもの以外)

- (初学者には推奨はしない) プログラミング言語C, カーニハン, リッチー, 石田晴久訳, 共立出版, 1989.
- (初学者には推奨はしない) ISO規格書 C11 規格番号 ISO/IEC 9899:2011 タイトルProgramming languages – C (日本規格協会からCD-ROMあるいはPDFファイルで入手可能。3-4万円程度。)

今回の講義内容

- 型変換
 - 暗黙の型変換（暗黙のうちに行われる型変換）
 - キャスト演算子（型変換を強制的に行う演算子）
- 条件分岐
 - if文
- 式の評価

準備

- TeraTermを起動し、oli.shibaura-it.ac.jpへログイン

プログラムのコンパイル、実行の手順(例)

- mkdirコマンドで自分の好きなディレクトリ(1kaiなど)を作成する。
\$ mkdir 1kai (\$はプロンプト。各自、設定により異なる)
- cdコマンドでそのディレクトリへ移動 \$ cd 1kai
- emacsコマンドでEmacsを起動 \$ emacs
- Emacs上でファイルを開く(C-x C-fの後ファイル名(test.cなど)を入力)
- プログラムを記述した後、保存(C-x C-s)
- Emacsを終了する(C-x C-c)か、あるいは中断する(C-z)
- gccコマンドでコンパイル \$ gcc test.c -o test
- 実行 \$./test
- test.cを再編集する場合は、Emacsをemacsコマンドでもう一度起動する(さきほど終了した場合)か、あるいはfgコマンドで再開させる(さきほど中断した場合)。

(参考) Terminalを2つ使って、1つはコンパイル用、もう1つはEmacs用にと、Emacsを終了したり中断したりする必要がなくなる。

例（打ち込んで実行）

```
#include <stdio.h>
int main (void)
{
    printf("test¥n");
    return 0;
}
```

注意 1： C言語プログラムのファイル名は.cで終わる名前にしてください。（gccがファイル名の拡張子部分を見るため）

注意 2： プログラムは（日本語部分以外は）半角英数字で記述する

改行について

さきほどのプログラムで、1行目以外は改行をしなくてもよい。例えば、以下のプログラムはさきほどのプログラムと同じ意味である。

```
#include <stdio.h>  
int main (void) { printf("test¥n"); return 0;}
```

[注意事項]

#includeの行は特別で、改行が必要である。

int, main, void, printf, returnなどの綴りの途中で改行してはいけない。

空白がある場所は自由に改行してよい（ダブルクォートの中、#includeの行は除く）。

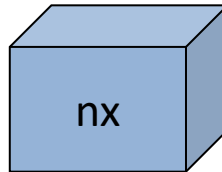
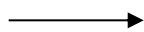
代入式における暗黙の型変換

int型の変数nxにdouble型の値9.99を代入する

```
int nx;  
nx = 9.99;
```

9.99

double型



int型

double型の9.99がint型の9に変換され、変数nxに代入される。

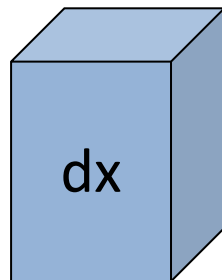
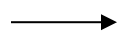
double型からint型へ変換される
ときは小数点以下が切り捨てら
れる。ただし、負の数の場合は0
に近い方へ切り捨てられる。

double型の変数dxにint型の値9を代入する

```
double dx;  
dx = 9;
```

9

int型



double型

9が浮動小数点数9.000...に変換され、変数dxに代入される。

四則演算における暗黙の型変換

int型同士、double型同士の演算

$$5 / 2 \rightarrow 2$$

$$5.0 / 2.0 \rightarrow 2.5$$

int型同士の場合結果もint型。
double型同士の場合結果もdouble型。

double型とint型の演算

$$5.0 / 2 \rightarrow 2.5$$

$$5 / 2.0 \rightarrow 2.5$$

片方がint型、もう片方がdouble型の場合、int型の数値をdouble型に変換してから演算が行われる（暗黙の型変換, implicit conversion）。+ や * などの演算でも同様。

四則演算における暗黙の型変換

四則演算 $+$, $-$, $*$, $/$ において、異なる型同士の演算では、
型が小さい方のオペランドは、大きい方の型に変換
された上で演算が行われる。

（注） $\%$ 演算子（割り算の余りを求める演算子）は
`double`型は引数にとれない。

数の型について

プログラム中に直接書かれている数の型は、2.0や2.5のように小数点が含まれている場合はdouble型、2や5などのように小数点がない場合はint型である。

キャスト演算子 --- 型を強制的に変換

(例1)

(double) 5 → 5.0

(int) 2.55 → 2

(int) -2.55 → -2

double型からint型へ変換するときは小数点以下が切り捨てられる。ただし、負の数の場合は0に近い方へ切り捨てられる。

キャスト演算子 --- 型を強制的に変換

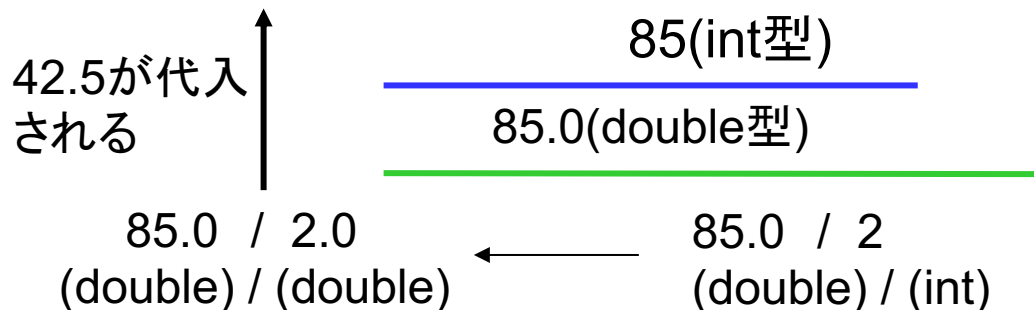
(例2)

```
double average;
```

```
average = (double) (40+45) / 2;
```

85は85.0にキャスト演算子(double)
によって強制的に型変換される

```
average = (double) (40 + 45) / 2 ;
```



2は2.0に暗黙に型変換
される(implicit conversion)

キャスト演算子の構文

構文

$(\langle \text{型} \rangle) \langle \text{式} \rangle$

式 $(t) e$ の意味

式 e の評価結果の値を型 t に変換した値が、式 $(t) e$ の評価結果となる。

例(打ち込んで実行)

```
#include <stdio.h>

int main (void) {
    double average;
    int intAver;
    average = (double)(40+45)/2;
    printf ("average=%f¥n", average);
    intAver = average;
    printf ("average=%d¥n", intAver);
    average = (40+45)/2;
    printf ("average=%f¥n", average);
    return 0;
}
```

if文による条件分岐

- 条件により、プログラムの流れを変える。

(例) キーボードから入力した整数値が、

- 5で割り切れなかったら、“5の倍数ではありません。”と表示する。
- そうでなかったら(5で割り切れたら)、“5の倍数です。”と表示する。

if文の構文(その 1)

if文の構文(その 1)

if (<式>) <文>

(これ自体も文である。たとえば、if文の中にif文を書くことができる。)

if文 **if (e) s** の意味

式eを評価し、その結果が0でないとき文sを実行する。

例(打ち込んで確認)

```
/* 読み込んだ整数値が5で割り切れるかどうか判定 */  
#include <stdio.h>  
int main(void)  
{  
    int x;  
    printf("整数を入力してください:");  
    scanf("%d", &x);  
    if (x % 5)  
        printf ("その数は5で割り切れません。¥n");  
    return 0;  
}
```

if文の構文（その2）

if文の構文（その2）

```
if (<式>) <文> else <文>
```

（これ自体も文である。たとえば、if文の中にif文を書くことができる。）

if文 **if (e) s1 else s2** の意味

式eを評価し、その結果が0でないとき文s1を実行し、0のとき文s2を実行する。

例（打ち込んで確認）

```
#include <stdio.h>
int main (void)
{
    int x;
    printf ("整数を入力してください:");
    scanf ("%d", &x);
    if (x % 5)
        printf ("その数は5で割り切れません。¥n");
    else
        printf ("その数は5で割り切れます。¥n");
    return 0;
}
```

式の評価

- C言語における式の例

- x
- 38
- $x + 38$
- $(x+38) / 2$

- 式の評価

- x がint型で、15が代入されていたとする。そのとき、
 - 式 x の値は15
 - 式 38 の値は38
 - 式 $x+38$ の値は、53
 - 式 $(x+38) / 2$ の値は、26
- 上記のように、複雑な式は、部分式の評価をまず行い、その結果の値を用いて演算(この例では足し算、割り算)を行う。

四則演算式

足し算式の構文 （引き算、掛け算なども同様。）

$\langle \text{式} \rangle + \langle \text{式} \rangle$

足し算式 $e1 + e2$ の意味

$e1$ と $e2$ をまず評価し、それらの結果 $v1$, $v2$ の和が足し算式 $e1+e2$ の評価結果である。

四則演算式は式である。したがって、たとえば $+$ の左側や右側に引き算式を書いてよい。

（例） $1 - 2 + 3$ など。この例では、赤字の部分の $1-2$ という式が、足し算式の左側の式を成している。

（補足）四則演算の演算子は左結合である。よって $1-2+3$ は $1-2$ に 3 を足すという意味である。 $2+3$ を先にしたい場合は括弧をつけ、 $1-(2+3)$ のように記述する。

値の比較の式: 等価演算子(その1) ==

構文

<式> == <式>

(これ自体も式である。たとえば、==の左辺や右辺で==を使った比較式を書ける。)

式 $e1 == e2$ の意味

式 $e1$, $e2$ を評価し、それらの結果 $v1$, $v2$ が等しいときは1、異なるときは0が、式 $e1 == e2$ の評価結果となる。

値比較のプログラム(1)

```
/* 二つの整数値をキーボードから読み込んで比較 */
#include <stdio.h>
int main (void)
{
    int x1;
    int x2;
    printf ("二つの整数を入力してください。¥n");
    printf ("整数1 :");
    scanf ("%d", &x1);
    printf ("整数2 :");
    scanf ("%d", &x2);
    if (x1 == x2)
        printf ("それらの値は同じです。¥n");
    else
        printf ("それらの値は違います。¥n");
    return 0;
}
```

値の比較の式：等価演算子(その2) !=

構文

<式> != <式>

(これ自体も式である。たとえば、==や!=の左辺や右辺で!=を使った比較式を書ける。)

式 $e1 != e2$ の意味

式 $e1$, $e2$ を評価し、それらの結果 $v1$, $v2$ が異なるときは1、等しいときは0が、式 $e1 != e2$ の評価結果となる。

値比較のプログラム(2)

```
/* 二つの整数値をキーボードから読み込んで比較 */
#include <stdio.h>
int main (void)
{
    int x1;
    int x2;
    printf ("2つの整数を入力してください。¥n");
    printf ("整数1:");
    scanf ("%d", &x1);
    printf ("整数2:");
    scanf ("%d", &x2);
    if (x1 != x2)
        printf ("それらの値は違います。¥n");
    else
        printf ("それらの値は同じです。¥n");
    return 0;
}
```

3つ以上の条件分岐をするには？

- これまでは流れを2つに分岐 → 3つ以上の場合は？
- 例) キーボードから読み込んだ整数値が

(1) 0

(2) 正

(3) 負

のいずれであるかによって処理を変えたい場合は、
if文の中でif文を使えばよい。(nesting, 入れ子)

復習 if文の構文(その2)

```
if (<式>) <文> else <文>
```

if文自体、文である。したがって、if文の中の
<文>のところにif文を書いてよい。

値比較のプログラム(3)(打ち込んで確認)

```
/* 読み込んだ整数値の符号を判定 */
#include <stdio.h>
int main (void)
{
    int  n;
    printf ("整数を入力してください:");
    scanf ("%d", &n);
    if (n == 0)
        printf ("その数は0です。¥n");
    else if (n > 0)
        printf ("その数は正です。¥n");
    else
        printf ("その数は負です。¥n");
    return 0;
}
```

赤字の部分はif文であり、外側のif文のelse部分を成している。

大小関係を比較する ～ 関係演算子 >

大小比較式の構文

<式> > <式>

大小比較式 $e1 > e2$ の意味

式 $e1$, $e2$ を評価し、その結果を $v1$, $v2$ とする。 $v1$ が $v2$ より大きければ1, そうでなければ0が、式 $e1 > e2$ の評価結果である。

この比較式自体、式である。 $==$ や $!=$ と同様、典型的にはif文の条件部分に使われるが、その他にも、式が書けるところなら自由に書くことができる。
比較演算子には、< 以外に、>, <=, >= もある。

大小関係を比較する ～ 関係演算子

比較式	意味
$e1 < e2$	式 $e1, e2$ を評価し、その結果を $v1, v2$ とすると、 $v1$ が $v2$ より小さければ1, そうでなければ0が、式 $e1 < e2$ の評価結果である。
$e1 > e2$	式 $e1, e2$ を評価し、その結果を $v1, v2$ とすると、 $v1$ が $v2$ より大きければ1, そうでなければ0が、式 $e1 > e2$ の評価結果である。
$e1 \leq e2$	式 $e1, e2$ を評価し、その結果を $v1, v2$ とすると、 $v1$ が $v2$ 以下のとき1, そうでなければ0が、式 $e1 \leq e2$ の評価結果である。
$e1 \geq e2$	式 $e1, e2$ を評価し、その結果を $v1, v2$ とすると、 $v1$ が $v2$ 以上のとき1, そうでなければ0が、式 $e1 \geq e2$ の評価結果である。

基本課題1

キーボードから金額をint型で読み込み、それに対し消費税を含む金額を計算するプログラムを書け。ただし、消費税は10%で、小数点以下は切り捨てとする。

[実行例]

```
[sasano@localhost enschu]$ ./kihon1-1
```

金額を入力してください: 100

消費税を含めた金額は110円です

```
[sasano@localhost enschu]$
```

(実行例において、赤字がキーボードからの入力部分である。)

[ヒント] 入力された数を1.1倍してからint型へ変換すればよい。(キャストによる明示的型変換あるいは代入式内の暗黙の型変換を使えばよい。)

基本課題2

int型の正の値をキーボードから読み込み、それが偶数か奇数かを判定し、結果を画面上に表示するプログラムを作成せよ。入力された値が0または負の数の場合は、正の数ではないと表示するようにせよ。

[実行例]

```
% ./kihon1-2
```

```
正の整数を入力してください: 100
```

```
100は偶数です
```

```
% ./kihon1-2
```

```
正の整数を入力してください: 99
```

```
99は奇数です
```

```
% ./kihon1-2
```

```
正の整数を入力してください: -3
```

```
-3は正の整数ではありません
```

```
%
```

発展課題1

西暦の年数(1868年以降)をキーボードから読み込み、明治、大正、昭和、平成、令和の表記に変換し、画面上に表示するプログラムを作成せよ。ただし、令和は永久に続くと仮定する。また、1年は元年と表示するようにせよ。境目の年（明治45年と大正元年、大正15年と昭和元年、昭和64年と平成元年、平成31年と令和元年）は、新しい元号で表示するようにせよ。

[実行例]

```
$ ./hatten2
```

西暦(1868年以降)を入力してください: 2000

2000年は平成12年です。

```
$ ./hatten2
```

西暦(1868年以降)を入力してください: 1912

1912年は大正元年です。

```
$
```

（上記実行例において、赤字がキーボードからの入力部分である。）

(ヒント) if文のネストで記述する。

発展課題2

- 西暦の年数（1582年以降）をキーボードから読み込み、閏年かどうかを判定し、結果を画面上に表示するプログラムを作成せよ。
- 閏年の定義としては、以下を用いる。
4で割り切れる年は閏年。
ただし、100で割り切れる年は閏年ではない。ただし、400で割り切れる年は閏年。

[実行例]

```
$ ./hatten2-2
```

西暦(1582年以降)を入力してください: 2010

2010年は閏年ではありません。

```
$ ./hatten2-2
```

西暦(1582年以降)を入力してください: 2000

2000年は閏年です。

```
$ ./hatten2-2
```

西暦(1582年以降)を入力してください: 1900

1900年は閏年ではありません。

参考課題1

int型の値を3つキーボードから読み込み、それらの最大値を画面に表示するプログラムを書け。

[実行例]

```
$ ./sankou2-1
```

3つの値から最大値を求めます。

1つ目の値を入力してください。:3

2つ目の値を入力してください。:4

3つ目の値を入力してください。:2

最大値は4です。

```
$
```

参考課題1 解答例

```
#include <stdio.h>
int main(void)
{
    int x;
    int y;
    int z;
    int max;
```

```
    /* 続き */
    max = x;
    if (max < y) max = y;
    if (max < z) max = z;
    printf("最大値は%dです。¥n" , max);
    return 0;
}
```

```
    printf("3つの値から最大値を求めます。¥n");
    printf("1つ目の値を入力してください。:");
    scanf("%d" , &x);
    printf("2つ目の値を入力してください。:");
    scanf("%d" , &y);
    printf("3つ目の値を入力してください。:");
    scanf("%d" , &z);
```

参考課題2

時速をキーボードから読み込み、秒速に変換して画面に表示するプログラムを書け。

[実行例]

```
$ ./sankou2-2
```

```
時速何kmですか: 100
```

```
それを秒速に変換すると秒速0.027778kmです
```

```
$
```

参考課題2 解答例

```
#include <stdio.h>
int main()
{
    double a;
    double b;
    printf("時速何kmですか:");
    scanf("%lf", &a);
    b=a/3600;
    printf("それを秒速に変換すると秒速%fkkmです", b);
    return 0;
}
```

参考課題3

double型の値をキーボードから受け取り、四捨五入した値を表示するプログラムを書け。

[実行例]

\$./sankou2-3

小数点第1位を四捨五入します。

値を入力してください: 2.6

四捨五入した値は3です。

\$

参考課題3 解答例

```
#include <stdio.h>
int main(void)
{
    double x;
    int y;
    printf("小数点第1位を四捨五入します。¥n");
    printf("値を入力してください:");
    scanf("%lf", &x);
    y = (int)(x+0.5);
    printf("四捨五入した値は%dです。¥n", y);
    return 0;
}
```

参考課題4

次のプログラムを実行すると1.000...と表示される。このプログラムを1.500...と表示されるようにせよ。

```
#include<stdio.h>
int main(void)
{
    printf("%f¥n", 3/2);
    return 0;
}
```

参考課題4 解答例

```
#include<stdio.h>
int main(void)
{
    printf("%f¥n",3.0/2);
    return 0;
}
```

上記の解答で、printfの部分は

printf("%f¥n",3/2.0);

や

printf("%f¥n",3.0/2.0);

でもよい。

参考課題5

身長をint型でキーボードから読み込み、それに対する標準体重をdouble型で求め、それを画面上に表示するプログラムを作成せよ。なお、標準体重の計算式は以下で与えられるものとする。

$$\text{標準体重} = (\text{身長} - 100) \times 0.9$$

[実行例]

```
$ ./kadai2-1
```

身長を入力してください: **176**

身長176cmの人の標準体重は68.400000kgです。

参考課題5 解答例

```
#include <stdio.h>
int main (void) {
    int height;
    printf ("身長を入力してください: ");
    scanf ("%d", &height);
    printf ("身長%dcmの人の標準体重は%fkgです。¥n",
           height, (height - 100) * 0.9);
    return 0;
}
```